

Chapter 15. 한계와 열린 문제

이 챕터에서 배우는 것 (이런 분이 먼저 읽으면 좋다: 지금까지 14개 챕터가 Book-forge 를 잘 작동하는 사례로만 그렸는데, 실제로는 무엇이 부족한지 정직한 평가가 필요한 분)

- Book-forge가 검증까지는 하지만 스스로 고치지는 못하는 이유
- "클라이언트 opt-in"이라는 설계가 어디까지 지켜주고 어디서부터 못 지켜주는지
- 이 책이 다루지 않은, 그러나 Book-forge 저장소에 이미 기록된 열린 항목들

15.1 검증은 하지만, 스스로 고치지는 못한다

10장에서 다룬 정적 검증기 3종도, 9장에서 다룬 Gate C(신뢰성) 낮은 점수 경고도, 전부 **결과를 보고할 뿐 그 결과를 근거로 다시 쓰지 않는다**. 저자가 낮은 Gate 점수나 정합성 검사 실패를 보고, 수동으로 `--force` 재생성을 트리거해야 한다. 이는 Book-forge 소스 코드 전체를 살펴봐도 확인되는 사실이다. 검증 결과를 LLM에게 "이걸 보고 스스로 고쳐 써라"라고 넘기는 경로가 어디에도 없다.

6장 (§ 6.1)에서 다룬 `review_loop.py` 가 정확히 이 능력에 가장 가깝다. 그러나 그 루프는 **사람이 피드백을 입력해야** 돈다. "검증기가 찾은 문제 목록"을 사람 대신 자동으로 넣어 1회 재생성하는 옵트인 경로는, 이 책을 쓰는 시점 기준으로 아직 존재하지 않는다. 만약 이 경로를 만든다면 무엇을 조심해야 할지는 이미 예측할 수 있다. 6장 (§ 6.2)의 `LoopDetectionConfig` 와 `MAX_REVIEW_ROUNDS` 같은 안전장치를, 사람이 개입하지 않는 자동 루프에도 똑같이(오히려 더 엄격하게) 적용해야 한다.

15.2 지식창고는 의도적으로 단순하다 — 그러나 한계가 있다

7장에서 다룬 `KnowledgeStore` 는 numpy 인메모리 코사인 유사도만 쓴다. 이 선택은 "별도 프로세스나 스키마 없이 로컬 파일 하나로 충분한 규모"라는 명시적 판단에서 나왔다. 결함이 아니라 **트레이드오프**다. 다만 이 트레이드오프가 항상 유리하지는 않다:

- 소스가 아주 커지면(수백 개 PDF 등) 인메모리 행렬 계산이 느려질 수 있다.
- 같은 챕터를 반복 `--force` 재생성할 때마다 임베딩을 다시 계산한다 — 쿼리 캐싱이 없다.
- 검색은 단일 쿼리 한 번뿐이다 — 다국어·동의어를 감안한 복수 쿼리 검색은 하지 않는다.

이 항목들은 실측으로 확인된 병목이 아니라 **아직 병목이 보이지 않은** 상태다. 지금 이 구조를 통째로 ChromaDB 같은 별도 벡터 DB로 바꾸는 것은 "로컬 파일 하나로"라는 원칙과 정면으로 배치되므로 권장되지 않는다. 저비용으로 얻을 수 있는 것(쿼리 캐싱)만 먼저 검토하고, 나머지는 실제 병목이 보일 때 대응하는 것이 Book-forge 저장소에 기록된 현재 방침이다.

15.3 이미지는 전적으로 저자의 몫이다

이 책 전체에서 다른 어떤 에이전트도 이미지를 자동으로 찾거나 배치하지 않는다. 저자가 마크다운에 `![alt](./images/xxx.png)` 를 직접 삽입해야 한다. 대안 이미지 검색, 위치 기반 자동 배치 같은 기능은 Book-forge의 핵심 가치(코드/구조 분석 기반 저술)와 거리가 있다고 판단해, 구현 여부 자체가 아직 재검토 대상으로 남아 있다.

15.4 "클라이언트 opt-in"의 한계 — 13장에서 예고한 지점

13장(§ 13.4)에서 이미 짚었다. `.aoo/claims.jsonl` 기반 팀 동시성 방어는 **저자가 자발적으로 `claims add` 를 실행해야** 작동한다. 서버가 강제하는 락이 아니다. 팀원 중 한 명이라도 이 관례를 지키지 않으면, 그 사람의 편집은 아무 보호도 받지 못한 채 다른 사람의 작업을 조용히 덮어쓸 수 있다. 이 경계는 Book-forge뿐 아니라 이 방식을 참고한 Agent-Evaluator SDK 전체의 정직한 한계이기도 하다. "서버 강제력이 필요한 시점"은 팀 규모가 커질수록 다가오지만, 지금의 Book-forge는 그 지점 이전의 도구다.

15.5 이 책이 다루지 않은, 그러나 저장소에 기록된 항목들

Book-forge 저장소의 `SPEC.md`에는 이 책이 직접 다루지 않은 열린 항목이 더 있다 — 정확성을 위해 목록만 남긴다(각 항목의 상세 논의는 `SPEC.md` 3부 참고).

항목	한 줄 요약
HTML 검색·스크롤스파이	완성된 HTML 산출물 안에서 전문 검색·읽는 위치 추적 UI 부재
커스텀 비주얼 컴포넌트 CSS	참고 자료 상자·팁 박스 이상의 시각적 컴포넌트 체계 부재
PDF 넓은 표 자동 축소	표가 페이지 폭을 넘으면 자동으로 줄이는 처리 부재
챕터 내 읽기 가이드·상호 참조	"이 절을 읽기 전에 § N을 먼저 보라" 같은 자동 상호 참조 부재
챕터 전용 독립 실행 예제 파일	실습 코드가 챕터 본문 안에만 있고 독립 실행 파일로 분리되지 않음
Part 서문 자동 생성	이 책처럼 사람이 쓴 Part 소개가 아니라, Part 단위 자동 생성 서문이 없음

이 목록을 이 챕터에 그대로 남기는 이유는 이 책 자체의 원칙(`00_기획안.md` § 2)과 같다 — **정확성이 핵심 가치인 책이라면, 다루지 못한 것도 숨기지 않고 명시해야 한다.**

15.6 명시적으로 안 쓰는 Config — "안 켜"도 하나의 결정이다

8장 (§ 8.6)의 관례를 뒤집어, "Book-forge의 14개 에이전트가 33개 Config 중 명시적으로 안 쓰는 것"도 정리해둔다 — 무엇을 켜는가만큼, 무엇을 일부러 안 켜는가도 이 프로젝트의 성격을 보여준다.

안 쓰는 Config/기능	관련 Gate	이유
자동 재작성 트리거	—	검증 결과를 근거로 스스로 다시 쓰는 경로 자체가 없다 (§ 15.1)
<code>ScopeConfig</code>	B	경로 검사는 직접 구현(12장 § 12.3) — 이 Config는 도구 이름 목록이라 애초에 용도가 다르다
이미지 검색/배치 관련 기능	—	Book-forge에 대응 기능 자체가 없다 (§ 15.3)

세 항목 다 "Config를 몰라서 안 컷다"가 아니라 "이 프로젝트 범위 밖이라고 판단해 안 컷다"는 점이 중요하다. `ScopeConfig` 는 특히 자주 오해되는 지점이다(12장 § 12.3에서 이미 짚었듯, 도구 이름 검사이지 경로 검사가 아니다). Config 하나를 "안 쓴다"는 결정도, "이 에이전트가 정확히 무엇을 하는가"를 좁게 규정하는 이 책 전체의 원칙(8장 § 8.6)이 그대로 적용된 결과다.

직접 해보기

이 책 전체에서 여러분이 점검해온 것(3장의 실패 유형 체크리스트, 8장의 Config 결정 트리)을 이제 반대 방향으로 뒤집어보자. 여러분의 에이전트(또는 앞으로 만들 에이전트)에서 "검증은 하지만 자동으로 고치지는 못하는" 지점이 어디인지, "클라이언트가 자발적으로 지켜야만 작동하는" 방어가 어디에 있는지 직접 목록으로 적어보라. 이 책이 15개 챕터에 걸쳐 Book-forge에 대해 한 일을 여러분 자신의 코드에 대해 해보는 것이 이 챕터, 그리고 이 책 전체의 마지막 실습이다.

이 챕터의 핵심

- **검증과 자동 교정은 다른 능력이다.** Book-forge는 전자만 갖췄다.
- **"의도적으로 단순한 설계"는 결함이 아니지만, 규모가 커지면 재검토가 필요할 수 있다.**
- **클라이언트 opt-in 방어는 팀 전체가 관례를 지킬 때만 작동한다.** 이 경계는 숨기지 않고 명시해야 한다.
- **이 책도 완결된 카탈로그가 아니다.** 저장소의 `SPEC.md` 가 이 책보다 더 최신의, 더 넓은 그림을 담고 있다.

참고 자료

- 부록 B(업계 동향) — 특히 B.2(에이전트 관측성)·B.4(Model Context Protocol, Book-forge가 채택하지 않은 인접 기술)
- `Book-forge/SPEC.md` — 3부(V-AE), 특히 AB·AC·AD·AE

- `src/book_forge/agents/review_loop.py` — 자동 교정 루프의 가장 가까운 전신
 - `src/book_forge/knowledge/store.py` — 지식창고 설계의 명시적 트레이드오프
-

이것이 마지막 챕터지만, 이 책 자체는 아직 한 걸음 남았다 — **맺음말**이 0장에서 펼쳤던 지도를 다시 펼쳐, 15개 챕터가 남긴 조각들을 하나로 모은다.